ONLINE RETAIL STORE

Data cleaning & EDA

By

Kedar Jadhav

PROJECT GOALS

- **Clean & Standardize:** Transform raw transactional data of 1,34,801 rows into a reliable, structured format using MySQL.
- **Discover Trends:** Analyze temporal sales patterns and overall monthly revenue.
- **Optimize Products:** Identify top-performing inventory and flag high-cancellation items.
- **Segment Customers:** Calculate Average Order Value (AOV) and execute an RFM (Recency, Frequency, Monetary) analysis to identify high-value buyers.

PROJECT STEPS

- **Keep Data Safe:** Made a copy of the original data first so nothing was lost.
- **Step-by-Step Work:** Split the project into Data Cleaning and Data Exploring.
- **Main Steps:**
- Backup Data
- Remove Duplicates & Errors
- Fix Text Casing & Formats
- Fill in Missing Info
- Analyze Sales & Products
- Score Customers (RFM) Recency, Frequency, and Monetary

Standardizing Data with String & CONCAT Functions

- Used **UPPER**, **LOWER**, and **TRIM** to fix messy text.
- Built a **CASE** statement to standardize remaining country names.
- **Result:** Clean, uniform categories ready for grouping.

```sql
UPDATE online_retail_data_2
SET country = CASE
    WHEN LOWER(TRIM(country)) = 'united kingdom' THEN 'United Kingdom'
    WHEN LOWER(TRIM(country)) = 'channel islands' THEN 'Channel Islands'
    WHEN LOWER(TRIM(country)) = 'united arab emirates' THEN 'United Arab Emirates'
    WHEN LOWER(TRIM(country)) = 'saudi arabia' THEN 'Saudi Arabia'
  WHEN LOWER(TRIM(country)) = 'czech republic' THEN 'Czech Republic'
    WHEN LOWER(TRIM(country)) = 'czech republic' THEN 'Czech Republic'
    ELSE country
END;
```

Country	CONCAT (UPPER(LEFT(Country,1)), LOWER(SUBSTRING(Country,2)))
Netherlands	Netherlands
Germany	Germany
Norway	Norway
EIRE	Eire
SWITZERLAND	Switzerland
Spain	Spain
Poland	Poland
Portugal	Portugal
ITALY	Italy
BELGIUM	Belgium
Lithuania	Lithuania
Japan	Japan
Iceland	Iceland
Channel Islands	Channel islands
denmark	Denmark
CYPRUS	Cyprus
Sweden	Sweden
finland	Finland
AUSTRIA	Austria
BAHRAIN	Bahrain
ISRAEL	Israel

```sql
SELECT DISTINCT Country,
CONCAT
(UPPER(LEFT(Country,1)),
LOWER(SUBSTRING(Country,2)))
FROM Online_retail_data_2;
```

RECOVERING DATA WITH SELF-JOINS

```
UPDATE online_retail_data_2 t1  -- SELF JOIN FOR POPULATING DATA
JOIN online_retail_data_2 t2
  ON t1.StockCode = t2.StockCode
SET t1.Description = t2.Description
WHERE t1.Description IS NULL
AND t2.Description IS NOT NULL
```

- **!!** Found missing product descriptions linked to known Stock Codes.
- Wrote a Self-Join to copy descriptions from clean rows into empty rows.
  - ***Result***: Saved **258 rows** of data from being deleted.

Sorting Returns vs. System Errors

* Used **LIKE** operator to find real cancelled orders (Invoice starts with 'C').

* Separated real returns from **303** "**Unknown Negative**" system errors.
* **Result:** Kept our final sales numbers highly accurate.

```
SELECT *,
CASE
  WHEN Quantity < 0 AND InvoiceNo LIKE 'C%' THEN 'Cancelled'
    WHEN Quantity < 0 AND InvoiceNo NOT LIKE 'C%' THEN 'Unknown_Negative'
    ELSE 'Normal'
END AS order_type
FROM online_retail_data_2 ;
```

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	row_num	OrderType
536589	21777	RECIPE BOX WITH METAL HEART	-10	2010-12-01	0.00	NULL	United Kingdom	1	Unknown_Negative
536764	84952C	MIRROR LOVE BIRD T-LIGHT HOLDER	-38	2010-12-02	0.00	NULL	United Kingdom	1	Unknown_Negative
536996	22712	CARD DOLLY GIRL	-20	2010-12-03	0.00	NULL	United Kingdom	1	Unknown_Negative
536997	22028	PENNY FARTHING BIRTHDAY CARD	-20	2010-12-03	0.00	NULL	United Kingdom	1	Unknown_Negative
536998	85067	CREAM SWEETHEART WALL CABINET	-6	2010-12-03	0.00	NULL	United Kingdom	1	Unknown_Negative
537000	21414	SCALLOP SHELL SOAP DISH	-22	2010-12-03	0.00	NULL	United Kingdom	1	Unknown_Negative
537001	21653	NULL	-6	2010-12-03	0.00	NULL	United Kingdom	1	Unknown_Negative

FINDING TOP WHALE CUSTOMERS (CTE & WINDOW FUNCTIONS)

- Used (**CTEs**) to organize the query and to find the **whale** customers.
- Used **Window Functions** to calculate a running total of revenue.
- **Result:** Proved that a tiny group of top customers brings in **80**% of our money.

```sql
WITH Customer_Totals AS (
    SELECT
        CustomerID,
        SUM(Quantity * UnitPrice) AS Individual_Revenue
    FROM eda_online_retail_data
    WHERE OrderType = 'Normal' AND CustomerID IS NOT NULL
    GROUP BY CustomerID
),
Running_Totals AS (
    SELECT CustomerID,Individual_Revenue,
        SUM(Individual_Revenue) OVER(ORDER BY Individual_Revenue DESC) AS
Running_Total,
        SUM(Individual_Revenue) OVER() AS Grand_Total
    FROM Customer_Totals)
SELECT
    CustomerID, Individual_Revenue,
    ROUND((Running_Total / Grand_Total) * 100, 2) AS Cumulative_Revenue_Pct
FROM Running_Totals
WHERE (Running_Total / Grand_Total) <= 0.80
ORDER BY Individual_Revenue DESC;
```

BUILD AN (RFM) SCORING MODEL

```
SELECT CustomerID,
DATEDIFF( (SELECT MAX(InvoiceDate) FROM eda_online_retail_data),MAX(InvoiceDate))AS
Recency_Days,
COUNT(DISTINCT InvoiceNo) AS Frequency_Orders,
SUM(Quantity * UnitPrice) AS Monetary_Spend
FROM eda_online_retail_data
WHERE OrderType = 'Normal'
AND CustomerID IS NOT NULL
GROUP BY CustomerID;
```

- **Recency:** Used **DATEDIFF** and **MAX** to count days since the last order.
- **Frequency:** Used **COUNT**(**DISTINCT**) to find total orders.
- **Monetary:** Used **SUM** and basic math (Quantity * UnitPrice) for total spend.

ACTIONABLE BUSINESS STRATEGIES

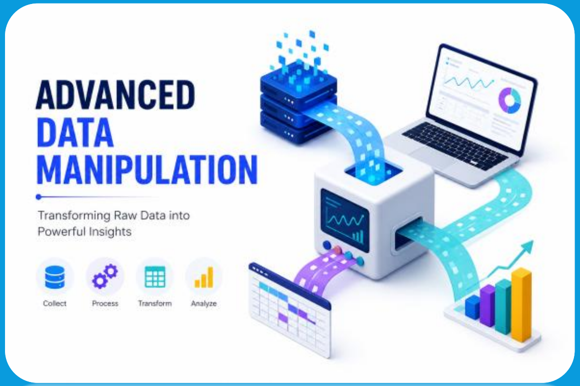
- **The Data:** 5% of customers drive 80% of our total sales.

- **The Action:** Create an exclusive **VIP Loyalty Program** for this segment. Offer early access to sales and dedicated support to ensure these high-value customers never leave for a competitor.

- **The Data**: Specific high-traffic products have unusual cancellation rates.
- **The Action**: Conduct an immediate **Quality Assurance (QA) Audit**. Check if the product descriptions are misleading or if the supplier quality has dropped, to stop refund losses.

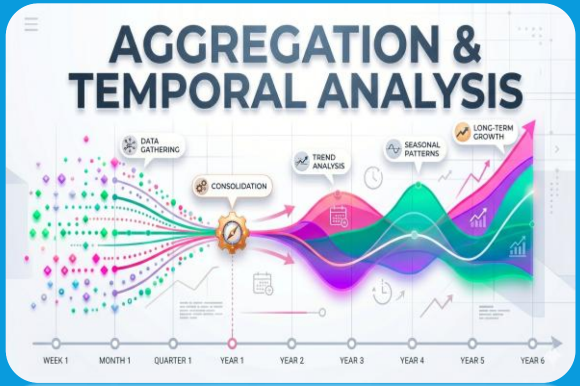
- **The Data**: We now have (RFM) scores for every buyer.
- **The Action**: Stop sending the same email to everyone. Send "Win-Back" discounts to high-spenders who haven't bought recently, and "Upsell" offers to frequent buyers.

Technical Skills Used



- **Window Functions:** Used **`ROW_NUMBER()`** for deduplication and **`SUM`**`()` **`OVER`**`()` for cumulative revenue calculations.
- **Common Table Expressions (CTEs):** Structured complex, multi-step queries (like the Pareto 80/20 analysis) using `WITH... AS` for clean, readable code.
- **Complex Joins:** Executed **Self-Joins** to intelligently fill in missing data based on matching relationship keys.

- **String Formatting:** Standardized messy text using **`TRIM`**`()`, **`UPPER(), LOWER(), CONCAT(),`** and **`SUBSTRING`**`()`.
- **Conditional Logic:** Built dynamic `CASE WHEN` statements to categorize order types and fix spelling errors.
- **Type Casting:** Converted raw text strings into usable formats using **`STR_TO_DATE`**`()` and **`DECIMAL`**`()`.



- **Date Math:** Calculated recency and time intervals using **`DATEDIFF`**`()` and **`DATE_FORMAT`**`()`.
- **Grouping:** Aggregated large datasets using **`GROUP`** `BY` and filtered aggregated results using **`HAVING`**.
- **Advanced analysis :** Used Aggregated functions to extract (RFM).

THANK YOU !

Open for questions, feedback, or suggestions.